passgen

Deterministic password generator

Documentation for project defense

Detailed description of all program functions

1. Project idea and problem being solved

In the modern world, every person has many accounts: email, social networks, banking, work, school. Each requires a password. Using the same password everywhere is very dangerous: if an attacker learns the password from one service (due to a database leak), they get access to all accounts. Remembering 20-30 different complex passwords is beyond human memory capacity.

Password managers (LastPass, Bitwarden) store all passwords in one encrypted place, but this creates a single point of failure. If an attacker gets access to the master password or the manager itself is hacked, all passwords are compromised.

passgen offers a fundamentally different approach — deterministic generation. The program does not store passwords but computes them each time using the cryptographic algorithm HMAC-SHA256. The user needs to remember only one phrase (seed). The password for any service is recovered by the formula: password = HMAC-SHA256(seed, service + length + step).

Advantages:
- Passwords do not need to be stored — they are always recoverable from seed and service name
- Nothing to steal — there is no password database on the computer
- Password cannot be lost — knowing the seed, it is recovered at any moment
- A unique password is created for each service
- No internet required
- Free and open source

2. Program structure (index.html)

The entire program is in one file — index.html. It consists of three parts: HTML (page markup), CSS (styles and design), and JavaScript (program logic). No additional files, libraries, or servers are required. The file opens in any modern browser.

2.1. HTML — page markup

The page consists of cards arranged vertically from top to bottom:
1. Top panel with "passgen" title and "Export"/"Import" links.
2. "Seed" card — secret phrase input and error message area.
3. "Service" card — service name input and "list" button.
4. "Settings" card — generation parameters.
5. "Generate" button.
6. "Login and password" card — generation result.
7. "Copy" button.
8. Import modal (hidden by default).

2.2. CSS — styles and design

Colors are set through CSS variables in the :root block. Main colors: light purple page background, white cards, dark text, purple accent. Buttons have gradient fill, cards have rounded corners and shadow. Animations: fadeUp (element appearance), glow (error border pulse), slideDown/slideUp (panel open/close), checkPop (checkbox toggle).

2.3. JavaScript — program logic

All program logic is written in JavaScript. It uses the Web Crypto API — a built-in browser cryptographic library available in Chrome, Firefox, Edge, Safari. It allows computing HMAC-SHA256 and SHA-256 directly on the user device without sending data to the internet. The code implements: password generation, saving and restoring settings for each service, JSON export and import, event handling (clicks, text input, keys).

3. Cryptographic algorithm

3.1. What is HMAC-SHA256

HMAC-SHA256 is a cryptographic algorithm that combines a hash function (SHA-256) with a secret key. HMAC stands for Hash-based Message Authentication Code. SHA-256 is a standard cryptographic hash function that transforms any input data into a 256-bit (32-byte) fixed-length value.

How is HMAC different from plain SHA-256? Plain SHA-256 simply takes input data and computes its hash. It works like a fingerprint: the original data cannot be recovered from the hash, but identical data produces identical hash. The problem is that anyone can compute SHA-256 of any data — no secret is needed.

HMAC solves this by adding a secret key into the hashing process. The algorithm mixes the key with the data using a special scheme (two hash passes with different key variants). The HMAC result cannot be computed without knowing the key. Even if an attacker learns the HMAC result, they cannot recover the key or forge the result for other data. In passgen, the user seed is used as this secret key.

In simple terms: if SHA-256 is a "black box" that turns any string into fixed-length output, HMAC-SHA256 is a "safe" that only opens with a special key (the seed). Without the key, computing the password is impossible. With the key, it is easy and always produces the same result.

3.2. Step-by-step password generation scheme

Step 1. The user enters the seed (secret phrase). The program converts this phrase into a set of bytes using the TextEncoder function. A cryptographic key is created from these bytes via crypto.subtle.importKey.

Step 2. The message for the algorithm is formed. The program takes the service name (e.g. "google"), adds the desired password length (e.g. 16) and the step number. The step number starts at 0 and increases each time the program needs more random data. All this is combined into one string and converted to bytes.

Step 3. HMAC-SHA256 is computed. The program calls crypto.subtle.sign("HMAC", key, message). The result is 32 bytes of cryptographically strong random data. These data cannot be predicted without knowing the key (seed).

Step 4. Bytes are converted to password characters. Each of the 32 bytes is checked: if its value is less than the maximum threshold (for uniform distribution), it becomes a character from the user-chosen alphabet. If the byte is unsuitable, it is skipped. If one block does not have enough suitable bytes, the step number increases and the next block is computed. The process repeats until the required number of characters is collected.

Step 5. Inversion options are applied. If the user enabled "First capital" — the first password character is made uppercase. If "Last digit" is enabled — the last character is replaced with a digit computed by a separate HMAC, to preserve the main password determinism.

3.3. Rejection Sampling — ensuring uniform distribution

When converting HMAC bytes to password characters, a mathematical problem arises. Each byte can have a value from 0 to 255 (256 values total). The alphabet length may be, for example, 50 or 62 characters. The number 256 does not divide evenly by 50 or 62.

If the program simply used byte % alphabet_length for all bytes, some characters would appear more often than others. For a 50-character alphabet: bytes 0-49, 50-99, 100-149, 150-199, 200-249 give uniform distribution (each character 5 times). But bytes 250-255 (6 of them) give characters 0-5 additionally. Result: characters 0-5 appear 6 times, characters 6-49 only 5 times. This bias is unacceptable in cryptography.

Solution: the program computes the maximum allowed value — the largest number divisible by alphabet length but not

exceeding 256. Formula: max = 256 - (256 % alphabet_length). Bytes with value >= max are simply discarded (rejected). Only bytes < max are used. Since max is a multiple of alphabet length, the character distribution is perfectly uniform.

Example: 50-character alphabet. max = 256 - (256 % 50) = 256 - 6 = 250. Bytes 250-255 are discarded. Bytes 0-249 give exactly 50 * 5 = 250 values. Each alphabet character appears exactly 5 times. Perfect uniform distribution.

This method is called Rejection Sampling. It is a standard technique in cryptography used in many password generation libraries, including well-known password managers.

3.4. Character set alphabets

Each character set contains a strictly defined list. Some sets exclude potentially confusable characters.

Uppercase: ABCDEFGHJKLMNPQRSTUVWXYZ (excluded I and O — confused with 1 and 0).

Lowercase: abcdefghikmnpqrstuvwxyz (excluded j, l, o — confused with digits).

Digits: 0123456789

Symbols: !@#$%&*+=-.

Defaults: uppercase + lowercase + symbols, length 10.

3.5. Login generation

Login is generated separately from password, using SHA-256 (no key, plain hash). The program takes the seed, adds the word "login", the service name and a counter value. The counter increases on each call, so each click on "Generate login" gives a new login.

From 32 hash bytes, the first 6 are taken and converted to characters from a special alphabet: "abcdefghjkmnpqrstuvwxyz23456789" (32 characters). Letters i, l, o (confused with 1 and 0) and digits 0, 1 (confused with letters) are excluded.

4. Program interface

Detailed description of each user interface element.

4.1. Seed field

This is the most important field in the program. The user enters their secret phrase here — the only thing to remember. Without the seed, the password cannot be recovered. If the seed is empty during generation, the field gets a pink pulsing border (glow animation) and shows the message "Enter seed". When typing begins, the error disappears. Pressing Enter in this field starts password generation.

4.2. Service field and service list

Enter the service name here: google, github, vk, yandex, etc. On the right side, there is a "list" button with an arrow that opens a panel with services. The panel shows popular services with colored logo circles and previously used user services.

Popular services: Google (blue G), GitHub (black GH), VK (blue V), Yandex (red Ya), Mail (blue @), iCloud (dark blue i), Onliner (orange O), Kufar (green K).

Clicking a service from the list fills the field and restores saved settings (length, character sets, inversion). If the desired service is not in the list, the user can type it manually — after generation it is automatically added to the list. Right-click on a saved service removes it from the list.

4.3. Generation parameters

Presets dropdown: "Minimal (8, letters+digits)", "Standard (10, everything except symbols)", "Maximum (16, everything)". "Custom" means manual settings. Password length from 1 to 99. Arrows for quick adjustment without keyboard. Four circles — character sets. Purple = enabled. Advanced settings: "First capital" and "Last digit" (hidden by default).

4.4. Generate button

Main button. Starts the full cycle: seed and checkbox validation, HMAC-SHA256, inversion, display, copying, adding to list, saving. Enter in seed/service fields equals clicking this button. Purple gradient with shadow.

4.5. Result

Login (read-only) + "Generate login" button. Password (read-only) — monospace, bold, centered. Indicator: 3 bars, color from pink to purple, text recommendation.

4.6. Copy button

Copies password to clipboard. Clipboard API, fallback execCommand. "Copied!" for 1.5s.

4.7. Export and import

Export: JSON with all services and their settings.
Import: loads JSON, shows selection modal with checkboxes.

5. All JavaScript functions (detailed description)

This section describes in detail each function of the program. The descriptions are written in human language, without using program code, so they can be orally retold during project defense.

5.0. Function call flow for password generation

Below is the sequence of function calls during password generation.

```
User clicks "Generate" or presses Enter
  |
  v
gen()
  |-- checks seed (if empty -> error, exit)
  |-- checks checkboxes (if none -> error, exit)
  |-- builds alphabet from selected sets
  |-- calls hmacGen(seed, svc, len, alpha)
  |     |-- importKey (seed -> HMAC key)
  |     |-- loop: sign (HMAC) -> byte selection -> character collection
  |     |-- returns password
  |
  |-- applies inversion (first capital, last digit)
  |-- writes password to pwd field
  |-- calls updateStrength(len, sets)
  |     |-- compares with thresholds (14/4, 10/3)
  |     |-- draws bars and text
  |
  |-- calls cpy() [clipboard copy]
  |-- adds service to svcs (if new)
  |-- calls renderSvcChips() [redraw list]
  |-- calls saveCfg() [save settings]
        |-- perSvc[service] = {state, inv, len}
```

Flow for login generation ("Generate login" button):

```
genLogin()
  |-- checks seed (if empty -> error, exit)
  |-- checks service (if empty -> exit)
  |-- loginCounter++
  |-- digest(SHA-256, seed+"login"+svc+counter)
  |-- 6 bytes -> characters from login alphabet
  |-- writes login to login field
```

5.1. Function init() — program initialization

This function runs automatically when the user opens the program page. It prepares all interface elements for work — sets initial values, attaches event handlers to fields and buttons.

First, init() calls loadState() to clear any data that might remain from previous use. Then it removes any errors from the seed field — clears the pink border and error message text.

The seed field gets an input handler: on each keystroke the error is removed. The seed and service fields get a keydown handler: pressing Enter starts password generation. The service field gets a blur handler: when focus is lost, the program remembers which service is selected and applies its settings.

Then init() checks if there are saved settings for the current service. If yes — restores them. If no — sets defaults: uppercase+lowercase+symbols, length 10, no inversion. Finally, it renders the service list with logos.

5.2. Function loadState() — complete data cleanup

Completely clears all program data so that each run starts with a clean slate. This is a key function for security.

It removes data from localStorage. Resets the perSvc settings object. Clears the svcs service array. Resets current service to "(empty)". Sets default settings, length 10. Clears the service input field.

Thanks to this function, if you close the program and open it again, no traces of previous use remain — not even service names.

5.3. Function saveCfg() — saving settings for a service

Remembers the current settings (checkboxes, length, inversion) and saves them for the current service. When the same service is selected again, settings are restored automatically.

It takes checkbox states from window.state, inversion options from window.inv, length from the input field. Packages everything into an object and saves in perSvc[serviceName]. Settings are in RAM only — they disappear when the page closes.

5.4. Function applyCfg() — restoring service settings

Applies saved settings for a selected service to the interface. Takes a cfg object, sets the length, restores checkboxes and inversion, redraws the interface.

5.5. Function toggleInv() — toggling advanced options

Toggles "First capital" or "Last digit" on click. Changes 0 to 1 and vice versa in window.inv. Updates the circle display. Does NOT save — saving only on "Generate".

5.6. Function applyPreset() — applying presets

Sets a ready combination of settings. Minimal: length 8, caps+lower+digits. Standard: length 10, caps+lower+digits. Maximum: length 16, all 4 sets. "Custom" — no action. After applying, redraws checkboxes and saves settings.

5.7. Function renderChk() — rendering character set checkboxes

Creates four rows with circles and set names. Active set gets a purple circle (class "on"). Click toggles the value and redraws.

5.8. Function renderInv() — rendering inversion options

Updates circles for cap and dig: adds or removes class "on".

5.9. Function adj() — changing password length

+1 or -1 on arrow clicks. Checks boundaries 1-99.

5.10. Function toggleSvcPanel() — opening/closing service list

Toggles class "open" on the panel. max-height 0 <-> 300px. Arrow rotates.

5.11. Function renderSvcChips() — building the service list

First shows popular services not yet in svcs, then all saved services from svcs. Calls chip() for each.

5.12. Function chip() — creating service button with logo

item can be an object (popular) or string (user). Colored circle with letter for popular, gray circle with "?" for user. Click: fills field, applies settings. Right-click: deletes.

5.13. Function gen() — password generation (main function)

Triggered by "Generate" button or Enter. Performs 10 steps:

1. Seed check — if empty, pink border, "Enter seed" message, stop.

2. Checkbox check — if none enabled, error, stop.

3. Alphabet assembly — from selected character sets.

4. hmacGen() — async HMAC-SHA256 computation.

5. Inversion: first capital (toUpperCase), last digit (separate HMAC).

6. Display password in pwd field.

7. updateStrength(len, sets) — strength indicator.

8. cpy() — clipboard copy.

9. Add service to svcs, sort, renderSvcChips().

10. saveCfg() — save settings.

5.14. Function hmacGen() — cryptographic program core

Performs password generation via HMAC-SHA256. Async (does not block browser).

seed -> bytes -> key via importKey. Message: svc+len in bytes. max = 256 - (256 % alpha.length) for rejection sampling.

Loop: while pwd.length < len:
- append counter ct to message
- crypto.subtle.sign("HMAC", key, message) -> 32 bytes
- for byte < max: char = alpha[byte % alpha.length]
- ct++

Returns the password.

5.15. Function genLogin() — login generation

6-character login. Checks seed and service. loginCounter++. SHA-256 of seed+"login"+svc+counter. First 6 bytes -> characters from "abcdefghjkmnpqrstuvwxyz23456789" (no i,l,o,0,1). Each click gives a new login.

5.16. Function updateStrength() — password strength indicator

len >= 14 and sets >= 4: strong (purple, 3 bars, "Excellent password"). len >= 10 and sets >= 3: medium (crimson, 2, "Good, could be longer"). Otherwise: weak (pink, 1, "Add symbols or increase length").

5.17. Function cpy() — copying password to clipboard

navigator.clipboard.writeText(), fallback execCommand("copy"). "Copied!" for 1.5 seconds.

5.18. Function exportData() — JSON export

Collects perSvc, forms data = {svcs, perSvc}, JSON.stringify with indentation, Blob, link, download = "passgen_export.json".

5.19. Function importData() — JSON import

FileReader, JSON parsing, checks perSvc/svcs fields, saves to importDataCache, showImportModal().

5.20. Function showImportModal() — import service selection

Service list with checkboxes (all checked=true). If empty — "No services to import".

5.21. Function confirmImport() — confirming import

Checked checkboxes -> perSvc[svc] = data, add to svcs, sort, redraw. "Import complete!" for 3 seconds.

5.22. Function closeImportModal() — closing import modal

remove class "open", importDataCache = null.

5.23. Function toggleAdvanced() — opening advanced settings

max-height 0 <-> 120px, opacity 0 <-> 1. Arrow rotates.

5.24. Function getCfg() — getting service settings

If perSvc[svc] does not exist — creates defaults. Returns settings.

5.25. Function onSvcChange() — service change handler

curSvc = field value, if settings exist — applyCfg().

5.26. Function saveState() — security stub

Empty. Previously saved to localStorage. Kept for compatibility.

6. Error handling

1. Empty seed — pink border, "Enter seed", clears on input.
2. No character set — "Select at least one character type".
3. Length out of 1-99 — forced boundary.
4. Invalid import file — error message.
5. Empty import file — "No services to import".
6. Clipboard unavailable — fallback execCommand.

7. Export and import (JSON format)

Example structure of export/import file:

```
{
  "svcs": ["google", "github", "vk"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

svcs — array of service names. perSvc — settings per service: state (flags for capitals, lowercase, digits, symbols), inv (cap — first capital, dig — last digit), len — password length.

8. Security

1. Locality. All computation happens locally in the browser. Data is not sent to the internet and cannot be intercepted.
2. Session mode. Settings in RAM only, everything is cleared on reload.
3. Standard cryptography. HMAC-SHA256 (RFC 2104). Used in banking, TLS, blockchain.
4. Rejection Sampling. Uniform character distribution, no modulo bias.
5. Irreversibility. Knowing the password, the seed cannot be recovered.
6. Different passwords — the service name is part of HMAC input.

Limitations

Does not protect against keyloggers. Does not require a master password (seed = master password). The seed must be stored separately.

9. Running the program

The program is a single file — index.html. How to run:

Open index.html in any modern browser (Chrome, Firefox, Edge, Safari, Opera) by double-clicking or through the browser menu (Ctrl+O -> select file).

System requirements:
- Any modern browser
- Web Crypto API (built into all modern browsers)
- No internet required